🚀 Guide Démarrage Rapide - Classification Factures Médicales

📋 Vue d'ensemble

Ce système utilise une approche **hybride Règles + Machine Learning** pour automatiser la classification des factures médicales :

- **Prédiction** : `status_verification` (validée / à_corriger / rejetée)
- **Explication** : `observations_verification` (commentaires automatiques)

Performance attendue : 92-95% accuracy, F1-score 93-96%

⚙️ Installation

1. Environnement Python

```bash
# Créer environnement virtuel
python -m venv venv
source venv/bin/activate  # Linux/Mac
# venv\Scripts\activate    # Windows

# Installer dépendances
pip install pandas numpy scikit-learn
pip install xgboost lightgbm catboost
pip install imbalanced-learn joblib
pip install matplotlib seaborn  # Pour visualisations
```

2. Structure Projet

```
projet-factures/
├── data/
│   ├── factures_historique.csv    # Données entraînement
│   └── factures_nouvelles.csv     # Nouvelles factures à classifier
├── models/
│   └── ensemble_factures.pkl      # Modèles entraînés (généré)
├── reports/
│   ├── confusion_matrix.png       # Visualisations (généré)
│   └── performance_report.txt
├── pipeline.py                    # Code principal
└── README.md
```

🎯 Utilisation en 3 Étapes

Étape 1 : Préparer vos Données

Votre CSV doit contenir **au minimum** ces colonnes :

```python
# Colonnes essentielles
nom_patient, age_patient, sex, visit_date
id_prescripteur, id_gerant
date_entree, date_sortie, nbre_jours

# Colonnes coûts/quantités
quantite_total_prod, quantite_total_act, quantite_total_ex
cout_total_prod, cout_total_act, cout_total_ex
cout_mise_en_observation, cout_evacuation

# Target (pour l'entraînement uniquement)
status_verification       # valeurs: "validee", "a_corriger", "rejetee"
observations_verification  # (optionnel, pas utilisé pour l'instant)
```

Étape 2 : Entraîner les Modèles

```python
import pandas as pd
from pipeline import full_pipeline

# Charger données historiques
df = pd.read_csv('data/factures_historique.csv')

# Vérifier distribution
print(df['status_verification'].value_counts())

# Entraîner
model_artifacts, X_test, y_test = full_pipeline(
    df_raw=df,
    train_mode=True
)

# Modèles sauvegardés automatiquement dans models/ensemble_factures.pkl
```

Durée : 5-15 minutes selon taille dataset (10K-100K factures)

Étape 3 : Prédire sur Nouvelles Factures

```python
import pandas as pd
from pipeline import full_pipeline

# Charger nouvelles factures (sans status_verification)
df_new = pd.read_csv('data/factures_nouvelles.csv')

# Prédire
predictions_df = full_pipeline(
    df_raw=df_new,
    model_artifacts=None,  # Charge depuis models/ensemble_factures.pkl
    train_mode=False
)

# Résultats
print(predictions_df[['index', 'status_verification', 'observations_verification']])

# Sauvegarder
predictions_df.to_csv('predictions_factures.csv', index=False)
```

Durée : ~0.1-0.5s par facture

📊 Évaluation Performance

Métriques Principales

```python
from sklearn.metrics import classification_report, confusion_matrix

# Après entraînement
y_pred = ensemble_predict(models, X_test, le_target)

print(classification_report(y_test, y_pred))

# Matrice de confusion
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

Visualisation

```python
```

```python
import matplotlib.pyplot as plt
import seaborn as sns

# Matrice de confusion
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['validee', 'a_corriger', 'rejetee'],
            yticklabels=['validee', 'a_corriger', 'rejetee'])
plt.title('Matrice de Confusion')
plt.ylabel('Vraie Classe')
plt.xlabel('Classe Prédite')
plt.savefig('reports/confusion_matrix.png', dpi=300)
```

🛠️ Personnalisation

1. Ajuster Règles Critiques

Modifier la fonction `apply_critical_rules()` dans `pipeline.py` :

```python
def apply_critical_rules(row):
    critical_issues = []

    # Ajouter vos règles spécifiques
    if row.get('cout_total_global', 0) > 50000:
        critical_issues.append("Coût exceptionnel > 50K - validation manuelle requise")

    # ... autres règles

    if critical_issues:
        return {
            'status_verification': 'rejetee',
            'observations_verification': ' | '.join(critical_issues),
            'source': 'hard_rules',
            'confidence': 1.0
        }

    return None
```

2. Optimiser Hyperparamètres

Utiliser Optuna pour tuning automatique :

```python
import optuna

def objective(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3),
    }

    model = XGBClassifier(**params, random_state=42)
    scores = cross_val_score(model, X_train, y_train, cv=5, scoring='f1_weighted')
    return np.mean(scores)

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=50)

print("Meilleurs paramètres:", study.best_params)
```

3. Améliorer Génération Observations

Pour une qualité supérieure, fine-tuner GPT-2 :

```python
```

```python
from transformers import GPT2LMHeadModel, GPT2Tokenizer, Trainer, TrainingArguments

# Préparer données d'entraînement
texts = []
for _, row in df.iterrows():
    context = f"""Status: {row['status_verification']}
Age: {row['age_patient_num']} ans
Coût: {row['cout_total_global']:.2f}
Observation: {row['observations_verification']}"""
    texts.append(context)

# Fine-tuning GPT-2
tokenizer = GPT2Tokenizer.from_pretrained('gpt2')
model = GPT2LMHeadModel.from_pretrained('gpt2')

# ... entraînement (voir code complet dans pipeline avancé)

# Utiliser pour génération
def generate_observations_llm(row, predicted_status):
    prompt = f"Status: {predicted_status}\nAge: {row['age_patient_num']}\n..."
    inputs = tokenizer(prompt, return_tensors='pt')
    outputs = model.generate(**inputs, max_length=100)
    return tokenizer.decode(outputs[0])
```

🔍 Monitoring Production

1. Détecter Data Drift

```python
from evidently.report import Report
from evidently.metric_preset import DataDriftPreset

# Comparer données actuelles vs référence
report = Report(metrics=[DataDriftPreset()])
report.run(reference_data=df_train, current_data=df_current)
report.save_html('reports/drift_report.html')
```

2. Feedback Loop

```python
```

```python
# Collecter feedback auditeurs
df_feedback = pd.read_csv('feedback_auditeurs.csv')

# Identifier erreurs modèle
errors = df_feedback[df_feedback['predicted'] != df_feedback['actual']]

# Ajouter aux données d'entraînement
df_enriched = pd.concat([df_train, errors])

# Réentraîner
model_artifacts_v2, _, _ = full_pipeline(df_enriched, train_mode=True)
```

3. A/B Testing

```python
# Envoyer 10% des factures à l'ancien système, 90% au nouveau
import random

for invoice in new_invoices:
    if random.random() < 0.1:
        result = legacy_system_predict(invoice)
    else:
        result = ml_system_predict(invoice)

    # Logger résultats pour comparaison
    log_prediction(invoice, result, system='legacy' if random.random() < 0.1 else 'ml')
```

📈 Optimisations Avancées

1. Gestion Classes Déséquilibrées

```python
```

```python
# Option 1: SMOTE (sursampling)
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
X_train, y_train = smote.fit_resample(X_train, y_train)

# Option 2: Class weights
xgb_model = XGBClassifier(
    scale_pos_weight=10,  # Pénalise davantage erreurs classe minoritaire
    ...
)

# Option 3: Focal Loss (pour déséquilibre extrême)
# Implémenter fonction de loss personnalisée
```

2. Feature Selection

```python
from sklearn.feature_selection import SelectKBest, f_classif

# Sélectionner top K features
selector = SelectKBest(f_classif, k=30)
X_selected = selector.fit_transform(X_train, y_train)

# Features les plus importantes
feature_scores = pd.DataFrame({
    'feature': X.columns,
    'score': selector.scores_
}).sort_values('score', ascending=False)

print(feature_scores.head(20))
```

3. Calibration Probabilités

```python
from sklearn.calibration import CalibratedClassifierCV

# Calibrer un modèle
calibrated_model = CalibratedClassifierCV(xgb_model, cv=5, method='sigmoid')
calibrated_model.fit(X_train, y_train)

# Probabilités mieux calibrées pour confiance
```

🐛 Dépannage

Problème 1 : Accuracy faible (<85%)

Causes possibles :

- Données déséquilibrées → Utiliser SMOTE ou class_weight
- Features peu discriminantes → Améliorer feature engineering
- Overfitting → Réduire max_depth, augmenter subsample

Solutions :

```python
# Analyser importance features
import matplotlib.pyplot as plt
from xgboost import plot_importance

plot_importance(xgb_model, max_num_features=20)
plt.tight_layout()
plt.show()
```

Problème 2 : Trop de faux positifs

Solution : Ajuster seuils de décision

```python
# Au lieu de argmax, utiliser seuils personnalisés
def predict_with_threshold(model, X, threshold_validee=0.7):
    proba = model.predict_proba(X)

    predictions = []
    for p in proba:
        if p[class_validee] > threshold_validee:
            predictions.append('validee')
        elif p[class_rejetee] > 0.8:
            predictions.append('rejetee')
        else:
            predictions.append('a_corriger')

    return predictions
```

Problème 3 : Lenteur en production

Solutions :

- Batch processing (traiter 100 factures à la fois)
- Caching features communes
- Modèle plus léger (LightGBM seul au lieu d'ensemble)
- Quantization des modèles

```python
# Batch processing
def predict_batch(invoices, batch_size=100):
    results = []
    for i in range(0, len(invoices), batch_size):
        batch = invoices[i:i+batch_size]
        batch_pred = full_pipeline(batch, train_mode=False)
        results.extend(batch_pred)
    return results
```

📚 Ressources

Documentation

- XGBoost : https://xgboost.readthedocs.io/
- LightGBM : https://lightgbm.readthedocs.io/
- Scikit-learn : https://scikit-learn.org/

Papers

- "Deep Learning for Invoice Processing" (ACM 2024)
- "Imbalanced Classification" (Springer 2023)

Datasets Publics (pour benchmarking)

- UCI ML Repository : Medical datasets
- Kaggle : Healthcare fraud detection

✅ Checklist Déploiement

- [ ] Données historiques nettoyées et validées
- [ ] Modèles entraînés et évalués (F1 >92%)

- [ ] Règles critiques testées et documentées
- [ ] API développée et testée
- [ ] Monitoring drift configuré
- [ ] Feedback loop implémenté
- [ ] Documentation utilisateurs créée
- [ ] Formation équipes réalisée
- [ ] Backup et rollback plan prêts
- [ ] Tests charge effectués (latence <2s)

🆘 Support

Pour questions ou problèmes :

1. Vérifier logs dans `logs/pipeline.log`
2. Tester sur petit échantillon (100 factures)
3. Valider format données d'entrée
4. Consulter matrice de confusion pour patterns d'erreurs

Contact : [votre email ou canal support]

Version : 1.0
Dernière mise à jour : Janvier 2025